

Class 8 ADC Task Program

```
/*
 * File:  main.c
 * Author:  sagar
 *
 * Created on 27 April, 2025, 3:09 PM
 */

#include <xc.h>
#include<stdint.h>
#define _XTAL_FREQ 2000000
// CONFIG
#pragma config FOSC = HS      // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF     // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = OFF    // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = OFF    // Brown-out Reset Enable bit (BOR disabled)
#pragma config LVP = ON       // Low-Voltage (Single-Supply) In-Circuit Serial Programming
                             // Enable bit (RB3/PGM pin has PGM function; low-voltage programming enabled)
#pragma config CPD = OFF      // Data EEPROM Memory Code Protection bit (Data EEPROM
                             // code protection off)
#pragma config WRT = OFF      // Flash Program Memory Write Enable bits (Write protection off;
                             // all program memory may be written to by EECON control)
#pragma config CP = OFF       // Flash Program Memory Code Protection bit (Code protection off)

/* Function prototypes */
void LcdInit(void);
void adcInit(void);
void LcdOutput(uint16_t i);
void LcdCommand(uint8_t i);
void LcdData(uint8_t i);
uint16_t map(long x, long in_min, long in_max, long out_min, long out_max);
void AN1_init(void);
void AN4_init(void);

/* Used variables */
uint16_t adcHigh, adcLow, calibratedValue, adcVolt;
char CH1[4] = "CH1:";
char CH4[4] = "CH4:";
```

```

/* main function */
void main(void)
{
    LcdInit();
    adcInit();

    LcdCommand(0x80); // First row
    for(int i = 0; i < 4; i++)
    {
        LcdData(CH1[i]);
    }

    LcdCommand(0xC0); // Second row
    for(int i = 0; i < 4; i++)
    {
        LcdData(CH4[i]);
    }
    while(1)
    {
        /* === AN1 Reading === */
        AN1_init();
        __delay_us(20);           // Acquisition delay
        ADCON0 |= 0x04;           // Start conversion (Set go-done)
        while(ADCON0 & 0x04);     // Wait for conversion
        adcHigh = ADRESH;
        adcLow = ADRESL;
        adcVolt = ((adcHigh << 8) | adcLow); // right justification

        LcdCommand(0x84);         // Display on first line after CH1:
        LcdOutput(adcVolt);
        __delay_ms(10);

        /* === AN4 Reading === */
        AN4_init();
        __delay_us(20);           // Acquisition delay
        ADCON0 |= 0x04;           // Start conversion (Set go-done)
        while(ADCON0 & 0x04);     // Wait for conversion
        adcHigh = ADRESH;
        adcLow = ADRESL;
        adcVolt = ((adcHigh << 8) | adcLow); // right justification

        calibratedValue = map(adcVolt, 0, 1023, 0, 135); // calibration
        if(calibratedValue >= 134) calibratedValue = 135;
        LcdCommand(0xC4);         // Display on second line after CH4:
        LcdOutput(calibratedValue);
    }
    return;
}

```

```

/* LCD init function */
void LcdInit(void)
{
    TRISC = 0x00; // Set PORTC as output (for control signals)
    TRISD = 0x00; // Set PORTD as output (for data signals)
    PORTC = 0x00;
    PORTD = 0x00;
    /****** LCD init *****/
    // LCD initialization sequence as per the HD44780 LCD datasheet
    LcdCommand(0x30); // Send function set command (8-bit mode)
    __delay_ms(100); // Delay for command execution
    LcdCommand(0x30); // Repeat function set command
    __delay_ms(100);
    LcdCommand(0x30); // Repeat function set command again
    __delay_ms(100);
    LcdCommand(0x38); // Set LCD for 8-bit mode, 2-line display, 5x8 font
    __delay_ms(100);
    LcdCommand(0x0C); // Turn on display, cursor off
    __delay_ms(100);
    LcdCommand(0x01); // Clear the display
    __delay_ms(100);
}

/* ADC init function */
void adcInit(void)
{
    TRISA = 0xFF; // All PORTA as input
    ADCON0 = 0x00; // Clear ADCON0
    ADCON1 = 0x89; // AN1 and AN4 analog, Vref = VDD, right justified
    ADCON0 |= (1 << 0); // Turn on ADC
    __delay_ms(10); // Stabilization delay
}

/* Channel 1 Selection function */
void AN1_init(void)
{
    ADCON0 &= 0b11000111; // Clear CHS bits
    ADCON0 |= (1 << 3); // CHS = 001 (AN1)
    __delay_us(20); // Acquisition time
}

/* Channel 4 selection function */
void AN4_init(void)
{
    ADCON0 &= 0b11000111; // Clear CHS bits
    ADCON0 |= (1 << 5); // CHS = 100 (AN4)
    __delay_us(20); // Acquisition time
}

```

```

/* value range map function */
uint16_t map(long x, long in_min, long in_max, long out_min, long out_max)
{
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

/* LCD output function */
void LcdOutput(uint16_t i)
{
    uint8_t d_0, d_1, d_2, d_3;

    if(i > 1023) i = 1023; // Clamp to max 135 for safety

    d_0 = i % 10;
    d_1 = (i/10)%10;
    d_2 = (i/100)%10;
    d_3 = (i/1000)%10;

    if(i < 10){
        LcdData(0x20);
        LcdData(0x20);
        LcdData(0x20);
        LcdData(0x30 + d_0);
    }else if(i < 100){
        LcdData(0x20);
        LcdData(0x20);
        LcdData(0x30 + d_1);
        LcdData(0x30 + d_0);
    }else if(i < 1000){
        LcdData(0x20);
        LcdData(0x30 + d_2);
        LcdData(0x30 + d_1);
        LcdData(0x30 + d_0);
    }else {
        LcdData(0x30 + d_3);
        LcdData(0x30 + d_2);
        LcdData(0x30 + d_1);
        LcdData(0x30 + d_0);
    }
}

```

```

/* Function to send data (characters) to the LCD */
void LcdData(uint8_t i)
{
    PORTC |= (0x1 << 3); // Set RS (RC3) = 1 (indicates data mode)
    PORTD = i;           // Place data on PORTD
    PORTC |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    __delay_ms(100);     // Small delay for command execution
    PORTC &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
}

/* Function to send commands to the LCD */
void LcdCommand(uint8_t i)
{
    PORTC &= ~(0x1 << 3); // Set RS (RC3) = 0 (indicates command mode)
    PORTD = i;           // Place command on PORTD
    PORTC |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    __delay_ms(100);     // Small delay for command execution
    PORTC &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
}

```